

Softwareontwerp 2: Eindopdracht

Ontwerpverantwoording

Business-laag

Domein

Aangezien deze applicatie niet geweldig ingewikkeld is, bestaat het domein maar uit twee klassen: User en Todo (zo genoemd om een conflict met het woord 'Task' te vermijden). Bij een taak hoort exact één gebruiker, bijgehouden in de vorm van het ID van die gebruiker.

Services

De twee services in deze laag zijn de messageservice en de todoservice. De Todoservice regelt communicatie met de repository, en stuurt ook waar nodig de messageservice aan als er veranderingen zijn doorgestuurd naar de repository die in de UI te zien moeten zijn. De message service is een wrapper van de WeakReferenceMessenger-klasse en stuurt dus allerlei berichten.

Verdere businesslaag

Ten slotte bevat deze laag ook nog allerlei soorten berichten om te versturen wanneer er veranderingen in de applicatie gebeuren waarvan de UI moet weten.

UI-laag

De UI gebruikt enkel MVVM en bevat zo goed als geen code in de codebehind. Elk venster heeft zijn eigen viewmodel(s) die INotifyPropertyChanged implementeren via een superklasse. Zo kan ik de viewmodels binden aan de view, die dan live aanpassingen aan waarden weergeeft. Enkele viewmodels implementeren ook IQueryAttributable om makkelijk parameters te kunnen ontvangen bij het navigeren naar dit scherm.

Ook maakte ik een speciale soort viewmodel, een 'ValidatableField<T>' en een aantal subklassen, voor velden die gevalideerd moeten worden. Deze velden houden zelf een waarde bij, valideren deze waarde, en regelen welke kleur de UI moet tonen indien een veld niet juist ingevuld is. Zo kan je dus makkelijk meerdere velden om te valideren toevoegen, het systeem is uitbreidbaar.

De vier categorieën van pagina's (en dus viewmodels) zijn: takenlijst, takendetail, gebruikerlijst en gebruikerdetail. Beide detailviewmodels houden ook een status bij: of er een nieuwe toegevoegd wordt of een bestaande aangepast. Afhankelijk daarvan worden bepaalde stukken code wel/niet uitgevoerd. Deze status is een enumeratie.

Dependency Injection

Zo veel mogelijk services en database-gerelateerde klassen werden geïnjecteerd via de MauiApp builder, om de verantwoordelijkheid van zulke objecten aan te maken uit handen te nemen van de viewmodels en/of andere klassen. De connectie naar de database werd als singleton geregistreerd om ervoor te zorgen dat er maar één connectie kan zijn.

Navigatie

Voor het navigeren tussen schermen heb ik me volledig gebaseerd op wat we in de lessen leerden. Een navigatieservice die geïnjecteerd wordt in de viewmodels, die alle navigatie bestuurt. De service is een wrapper voor de `Shell.Current.GoToAsync` functionaliteit. Alle navigatieverantwoordelijkheid ligt bij deze klasse, en er kunnen dus makkelijk aanpassingen aan gemaakt zodat het hele systeem mee aangepast wordt. Bij het navigeren is er de optie om parameters mee te geven naar het doelscherm.

Commands

Ik voegde zelf nog een 'AsyncCommand' toe die gemakkelijk een asynchrone methode kan bevatten in plaats van enkel gewone methodes.

Sorteren en filteren

Voor zowel sorteren als filteren van taken is er gebruik gemaakt van een strategy pattern. Voor sorteren zijn de strategies gebaseerd op de bestaande `Icomparer` interface, voor filteren zijn ze gebaseerd op een eigen interface. Op deze manier kan men makkelijk nieuwe filter- en/of sorteropties toevoegen.

Datalaag

Database en repository

Als database werd een LiteDB database gebruikt. Hiervoor baseerde ik me ook op de code op de github-repository van de lector. Vooral het idee om de databaseconnectie te scheiden van de repository, en beide individueel toe te voegen via dependency injection haalde ik daar.

De repository zelf is een simpele wrapper voor de LiteDB library.